BUSINESS & PRODUCT BLUEPRINT

DungeonFlux

The hosted business, from first login to saved adventure.

A detailed plan for shared session funding, transparent metered billing, DigitalOcean hosting, persistent characters, private media, and recoverable campaigns.

BUILD THE TRUST LAYER

Identity and consent · Money and metering
Files and saves · Operations and recovery
Pages and journeys · Launch acceptance

Prepared for Earl Cameron

Planning date: October 24, 2026 · America/New_York
Repository baseline: 7350b151e863
Proposed implementation specification. Pricing and launch policies require validation.

Contents

A decision-oriented commercial plan and an implementation-ready product specification.

1. Executive decisions 3

2. Business strategy and economics 4

3. Product vocabulary and permissions 6

4. Customer pricing and spending contract 8

5. Funding, group pots, and payment flows 10

6. Authoritative usage billing and accounting 12

7. Billing governance and customer protection 15

8. Information architecture and page inventory 16

9. Detailed page specifications 17

10. End-to-end user journeys 21

11. Notifications and UX behavior 23

12. DigitalOcean hosting plan 24

13. File storage, player characters, and saved campaigns 27

14. Security, privacy, content, and commercial readiness 32

15. Domain model and engineering contracts 33

16. Observability, performance, and operations 36

17. Acceptance test catalog 37

18. Implementation sequence and work packages 39

19. Launch decision and operating scorecard 41

20. Source register and evidence boundaries 42

How to use this plan

Sections 1–4 define the offer and commercial rules. Sections 5–8 define money, accounting, and consent. Sections 9–11 specify pages and user journeys. Sections 12–14 define hosting, files, saves, and security. Sections 15–19 define implementation contracts, launch gates, operations, and execution order. The final section lists sources and unresolved decisions.

All prices proposed for customers, quotas, latency targets, and rollout thresholds are **testable product decisions**, not vendor guarantees. Vendor prices are identified separately. Numbers exclude applicable taxes unless expressly stated. Payment processing approval and legal/accounting review remain launch gates.

1. Executive decisions

1.1 What to build first

Launch an invite-only, browser-based hosted pilot for adult groups, initially in the United States and billed in USD. One host creates a campaign; teammates join through a link or room code, bring saved characters, and may contribute to a shared session allowance. The host does not receive money and cannot withdraw another person's contribution.

The public offer is a game service, not an AI-token reseller, financial wallet, peer-to-peer payment system, or marketplace. Customer funding is usable only for DungeonFlux services, subject to the exact payment model approved by the processor and counsel. A different label does not change the legal substance.

The first release should include one well-tested adventure or bounded free-roam experience, two to six authenticated players subject to load qualification, a paired display, saved characters, durable campaign progress, transparent usage, manual funding, and refunds to original payers. The supported rules subset must be accurately disclosed. Broader rules coverage remains a roadmap goal, not a launch claim.

1.2 Recommended decisions

Decision	Initial choice	Why
Compute	DigitalOcean Basic Droplets	Already owned account; native Rust support; predictable starting cost.
Database	DigitalOcean Managed PostgreSQL	Canonical game state, identities, financial ledger, metadata, and outbox jobs.
File storage	DigitalOcean Spaces, private by default	Portraits, generated media, exports, immutable checkpoint bundles, and backups.
Authentication	WorkOS AuthKit hosted flow, subject to integration validation	Avoid owning passwords, account recovery, and social-login security.
Payments	Stripe hosted Checkout for one-time purchases, subject to written business-model approval	Card collection stays outside DungeonFlux; custom service ledger remains authoritative.
Revenue	Test $2 per active table-hour plus 1.5× the published provider tariff for customer-billable AI work	Separates platform value from variable generation consumption.
Funding	Source-attributed allocations to a session	Group budget without transferring ownership to a host.

Decision	Initial choice	Why
Defaults	Manual top-ups; premium video off; conservative media frequency	Reduces accidental charges and difficult support cases.
Release process	Immutable builds, CI, staging, canary sessions, measured launch gates	Acceptance evidence matters more than commit count.
Deployment shape	Control plane plus isolated campaign runtimes on a small worker pool	Current runtime is not yet proven safe for many tenants in one process.
Deferred	Subscriptions, auto-reload, cash transfers, creator marketplace, public matchmaking	Avoids unnecessary complexity before paid-session retention is demonstrated.

AuthKit currently advertises free user management through one million monthly active users; customization and other WorkOS services can cost extra. Hosted authentication supports the essential sign-up, sign-in, verification, recovery, and MFA flows. [S10–S11]

1.3 Current implementation is not the finished business

The refreshed README explicitly says there is no hosted service, account system, or billing implementation. The current game runs through `df-tools`; the production `df-server` is not a workspace member. Existing `df-commerce` types contain valuable proposal logic, but a proposal is not a durable reservation, settled charge, or payment receipt. Reuse those types without mistaking them for a live commerce system. [R1–R2]

The new image-spend note reports approximately $45.60 in testing spend, but does not establish the number of comparable successful table-hours. Therefore, the earlier roughly $1.08 three-hour non-video estimate must not be used as production cost of goods sold. [R3]

2. Business strategy and economics

2.1 Customer and positioning

The first customer is an existing group that already organizes game nights and wants an easier way to play together without a dedicated expert game master. The buyer is usually the organizer, but the funding design allows friends to share costs without reimbursement awkwardness.

Sell the outcome: a memorable group adventure, persistent characters, cinematic presentation, understandable consequences, and predictable spending. Do not sell the number of models, crates, or generated images. The distinctive product hypothesis is the combination of a shared display, private player views, persistent adventures, and cooperative budgeting—not an unsupported claim that competitors lack AI narration.

The initial sales motion is founder-led. Recruit a small number of real groups; observe setup and complete sessions; fix failures before broad promotion. Encourage a participant to host a later campaign. Introduce referral bonuses only after fraud-resistant activation events exist.

2.2 Revenue model

Use a versioned price schedule:

`retail service charge = active table time × platform hourly rate + customer-billable provider units × pinned provider tariff × markup`

Pilot price hypothesis:

- Platform rate: **$2 per active table-hour**, shared by the entire table.
- AI retail rate: **1.5× a published, pinned supplier tariff**, with transparent categories.
- No mandatory per-player joining charge.
- No minimum one-hour charge; use exact time metering.
- Premium cinematics receive explicit quotes and approvals.
- Routine saves, viewing existing characters, viewing paid assets, and ordinary export do not trigger new AI charges.
- A genuinely new portrait, scene, or regenerated output is quoted or included in the approved automatic-generation policy.

The tariff is a customer price basis, not a promise to expose every negotiated vendor discount. It must say which inputs count: text input/output, reference-image input, audio duration, image output, video duration, and any paid analysis calls. Freeze a price version for a session. A provider price change does not silently reprice an active session.

Do not bill users for development prompts, coding-agent subscriptions, internal debugging, failed validations caused by a defect, duplicate requests, or speculative assets that are never used under the proposed consumer policy. Track those as company expense. Customer-selected retries or alternate versions can be billed only after disclosure and consent.

2.3 Your stated development budget

Expense	Monthly planning amount
Claude Code subscription, as stated	$200.00
Codex subscription, as stated	$100.00
Development API budget, $100/week × 52/12	$433.33
Development baseline	**$733.33**

These are your supplied inputs, not newly verified vendor plan quotations. The $100/week is treated as research and development, not customer production usage. If that budget already includes production usage, reclassify the invoices rather than counting the same cost twice. Hosting, legal work, business administration, customer acquisition, refunds, taxes, and owner compensation are additional.

2.4 Unit economics under uncertainty

The following is a sensitivity model, not a forecast. Each row represents a three-hour table session. Payment fees are allocated at 4.4% of consumed retail value, corresponding to the current 2.9% + $0.30 domestic-card fee on a fully consumed $20 purchase. A $1 per-session variable support/incident reserve is assumed. Production infrastructure is recovered separately as a fixed expense. [S8]

AI supplier cost	Table revenue: $6 + 1.5× cost	Allocated payment fee	Support/ incident reserve	Contribution before fixed expenses	Contribution margin
$1.00	$7.50	$0.33	$1.00	$5.17	68.9%
$4.00	$12.00	$0.53	$1.00	$6.47	53.9%
$10.00	$21.00	$0.92	$1.00	$9.08	43.2%
$25.00	$43.50	$1.91	$1.00	$15.59	35.8%

A higher media bill increases absolute contribution in this model but lowers percentage margin and raises customer sticker shock. It is not automatically a better business. Failed-but-billed work, refunds, international cards, disputes, taxes, and additional support can materially worsen these figures.

With the $4 supplier-cost scenario and a $100/month pilot hosting envelope, roughly **129 three-hour sessions per month**, or **30 groups playing weekly**, cover $833.33 of development plus hosting. This excludes salary, acquisition, legal work, and other fixed expenses. With a $380 production hosting envelope, the corresponding baseline is $1,113.33 and roughly 173 sessions, or 40 weekly groups.

At 100 weekly groups, the same hypothetical model yields about $5,200/month in consumed service revenue and $2,804.53 contribution before fixed expenses. After the $833.33 baseline, $1,971.20 remains before owner pay, other operating expenses, acquisition, and taxes. This is a scenario, not evidence of demand or profit.

2.5 How to validate the offer

Run a closed cohort of approximately 10–20 groups and at least 30 complete sessions across supported party sizes. Collect actual provider invoices, successful-output counts, time spent in gameplay versus waiting, support minutes, refund reasons, and second-session behavior.

Suggested experimental gates: at least 90% successful onboarding, at least 95% session completion without engineering intervention, no financial integrity errors, and at least 50% of activated groups voluntarily returning within 30 days. These are product targets, not statistically proven market thresholds. Report counts and confidence limitations.

Test pricing at the table level, not through hidden individualized pricing. Show the same price version to all contributors in one session. A later host subscription could reduce the platform hourly fee or add storage, but should not promise unlimited generation.

3. Product vocabulary and permissions

3.1 Distinct objects

Account: A person's identity, private character library, payment sources, credits, security settings, and data preferences.

Campaign: A persistent world and story with members, character instances, saved progress, assets, and rules/content versions. It exists even when no session is running.

Session: One scheduled or active play interval within a campaign, with a specific price version and funding policy.

Table: The currently connected participants and paired shared display. One campaign has at most one authoritative active session in the initial release.

Account credit: A source-attributed claim to future DungeonFlux services, only if this purchase structure is approved. It is not transferable cash.

Session allocation: A payer's revocable authorization to spend a specified part of their available credit in one session, except amounts already reserved or consumed.

Pot: The user-facing sum of those allocations. It is not a new financial owner.

Reservation: The maximum retail charge temporarily held for an admitted operation; it is distinct from a final charge.

Character template: A reusable player-owned identity and build. **Campaign character:** The authoritative campaign-specific instance with progression and inventory.

3.2 Role matrix

Capability	Host	Co-host	Player	Contributor without seat	Paired display	Support
Invite/remove players	Yes	Delegated	No	No	No	Audited assistance only
Start/pause session	Yes	Delegated	Emergency pause request	No	No	Incident control
Allocate own credit	Yes	Yes	Yes	Yes	No	Never impersonate consent
Raise another payer's exposure	No	No	No	No	No	No
View shared pot totals	Yes	Yes	Yes	Yes	Optional summary	Scoped, audited
View own card/ receipts	Own only	Own only	Own only	Own only	Never	Redacted support data
Edit own character template	Yes	Yes	Yes	Yes	No	Recovery assistance
Rewrite authoritative campaign inventory	Through legal game actions	Through legal game actions	Through legal game actions	No	No	Exceptional audited correction
View private character notes	Own only	Own only	Own only	Own only	Never	Break-glass, justified
Export campaign	Audience-filtered	Delegated	Own permitted view	Funding records only	No	Backup role separately
Delete campaign	Host with confirmation	Not by default	Leave only	No	No	Policy-controlled

Payment and campaign roles are independent. Someone can sponsor a session without becoming a game host. A host cannot see a payer's total personal credit balance, full billing address, or another campaign's activity.

4. Customer pricing and spending contract

4.1 State-driven time billing

Use the authoritative server state, not browser animation or an open WebSocket, to decide whether platform time accrues.

Session state	Platform time	New paid AI requests	Saving and viewing
Draft / scheduled / lobby	Free	Only separately approved setup generation	Available
Preparing	Free	Approved preparation budget only	Available
Active	Metered	Within every approved cap	Available
Paused by host	Free	No new background generation	Available
Waiting on a blocking provider failure	Free for blocked interval	Bounded recovery policy only	Available
Low balance	Metered while useful play continues	Optional work reduced; essential work reserved	Available
Funding required	Free	No new paid work	Available
Reconnecting / unhealthy owner	Free when table cannot use the service	No unowned work	Recovery available
Settling / ended	Free	No new generation without a new explicit request	Available

A provider wait does not pause the entire meter when the table can meaningfully continue other play. Define a blocking-wait signal in the game runtime and expose it in the UI. Long pauses must not secretly continue prefetching. If all interactive players disconnect, stop new work immediately and stop time billing promptly; a proposal is a 30-second presence detection bound with an automatic credit for that unplayable interval.

Meter active time with a rational accumulator. At $2/hour, do not round every second or minute up to a cent. Maintain integer micro-dollar accounting and carry division remainders. Round only at documented receipt/refund boundaries, consistently and without charging more than the applicable cap.

4.2 Caps are ceilings, not targets

Expose four independent controls:

1. **Table hourly ceiling:** Maximum admitted retail spend in a rolling 60-minute window, including the platform fee and applicable consumption-time tax.
2. **Session ceiling:** Maximum total retail spend for this session, including separately approved setup work.
3. **Individual authorization:** Maximum a given payer can lose in this session.
4. **Per-request threshold:** Large individual jobs require an explicit quote/approval.

In-flight reservations count against ceilings. A long-running reservation never becomes free merely because it is older than the rolling window. Settled charges and open holds must not be double-counted. Lowering a cap immediately prevents additional work; it cannot retroactively cancel already-sent provider jobs. The UI shows outstanding commitments before confirming a decrease.

In addition to the hard hourly ceiling, use a pacing controller to avoid spending the entire hour's budget in the opening minute. Reserve capacity for the next platform billing interval and essential narration before optional pictures. A bounded preparation allowance can fund initial portraits and scene setup, but it must be displayed separately and remain within the session and payer ceilings.

No backend worker can bypass these limits. Model-selected tool calls, prefetch, retries, moderation/quality checks, and memory maintenance all pass through the same supplier-admission boundary, even when the company—not the customer—pays.

4.3 Presets

Preset	Proposed table ceiling	Intended behavior
Story	$3/hour	Economical narration, text-first interaction, reused art; no automatic video.
Immersive	$8/hour	Voice and periodic images within the remaining allowance.
Cinematic	$20/hour	More frequent visual updates; expensive video still explicitly approved.
Custom	Customer-selected	Category controls, per-job threshold, and total session ceiling.

Presets are not guarantees that every model or media feature fits at every rate. A $3 ceiling leaves only $1/hour of retail AI budget after the proposed platform fee. If acceptable gameplay cannot operate within that constraint, change the preset or disable it rather than silently overspending.

Show runway as a range based on recent usage and available funding. Label `at selected maximum` separately from `at your recent rate`. An hourly ceiling allows bursty consumption, so balance divided by rate is not a promise that a particular number of useful hours remains.

4.4 Customer-visible generation policy

The table approves categories and frequency, not raw API credentials. Minimum controls: spoken narration, microphone transcription, generated scene stills, character portraits, music, and optional cinematics. A control can say `Use existing art only` rather than making customers reason about cache implementations.

Default: no video; one active scene image job at a time; reuse approved character references; cap scene refresh cadence; suppress unused speculative requests. An automatic image mode displays a representative quote and maximum frequency before consent.

A server or quality failure is not transformed into character dialogue about insufficient knowledge. Surface `Narration provider unavailable; no new charge` and provide a functional fallback or pause.

5. Funding, group pots, and payment flows

5.1 Processor and legal gate

Obtain written approval for the precise proposed arrangement: seller-issued, USD-denominated service credit; multiple purchasers allocating their own credit to one game session; no cash withdrawal, peer transfers, gambling, or third-party merchant redemption. Submit flow diagrams, balance limits, refunds, age policy, generated-content policy, and dispute handling.

Stripe's service terms prohibit using **Stripe Billing's credits feature** for stored-value/wallet functionality. Its restricted-business policy also addresses stored value and credits. A custom ledger does not exempt the product from these requirements. [S6–S7]

Do not use Stripe Connect unless the business later pays independent sellers or game masters. The initial company is the sole seller. Every contribution is a purchase from that company, not a transfer to a session host.

If reusable credit is not approved, the fallback is separately approved, fixed-price session-service purchases by each contributor, with unused-service refunds according to the disclosed policy. Do not ship the same transferable wallet under a different name. This fallback changes the UX and must be reflected in every screen.

5.2 Funding purchase

Recommended starting presets: $10, $20, and $50. Prefer $20 as the default to reduce the fixed processing-fee impact; do not force a large purchase merely to test the game. Initial risk-policy limits might be $100 per purchase, $200 outstanding purchased credit, and $200/day per new payer. These are proposed operational controls, not legal safe harbors.

Flow:

1. Authenticated payer selects amount and sees credit value, taxes if applicable, total bank charge, refund terms, and merchant identity.
2. Backend creates a purchase order and hosted Checkout Session with an idempotency key.
3. Customer pays on the processor's hosted page. Card data never enters DungeonFlux servers.
4. Return page says `Confirming payment`; it does not mint credit from a redirect URL.
5. A signed webhook is stored durably, deduplicated, and processed. Retrieve authoritative payment status when needed.
6. On confirmed success, post a balanced ledger entry and activate the funding lot. Only then increase spendable credit.
7. Deliver a receipt and offer to allocate some or all to the intended session.

Stripe explicitly recommends webhook-driven fulfillment; deliveries may be duplicated or out of order. Handle both event-ID duplication and duplicate business effects for the same payment object. [S9]

Payment states: `Created → RequiresAction / Processing → Succeeded → PartiallyRefunded / Refunded / Disputed`, with independent failure/cancellation outcomes. A pending payment is never displayed as spendable. The purchase remains discoverable after browser closure.

5.3 Allocation to a session

The payer chooses a maximum contribution, approves the price version, hourly ceiling, session maximum, and host's permitted generation settings. Record the consent version and a timestamp. Allocation moves available credit into an encumbered sub-balance; it does not create service revenue or transfer ownership.

The pot displays **allocated**, **reserved**, **spent**, and **available for new work**. Each payer sees their own detailed values. Shared contributor names/amounts may be visible by explicit group setting; card details and total personal account balances never are.

Default split: proportional to contributors' authorized amounts within a funding epoch. For example, 20/10/$10 authorizations produce 50%/25%/25% of new shared charges. Changes establish a new epoch for future requests. Save the exact allocation on each reservation; never retrospectively redistribute a completed charge.

Offer host-sponsored and equal-share modes only with clear payer consent. Do not initially implement complicated priority cascades, debt between friends, or automatic reimbursements.

5.4 Example: three contributors

Contributor	Allocated	Share of an $8 settled session charge	Unused allocation
Host	$20.00	$4.00	$16.00
Player B	$10.00	$2.00	$8.00
Player C	$10.00	$2.00	$8.00
Total	$40.00	$8.00	$32.00

On end, unused allocations return to each contributor's available service credit, not the host. This is an internal release, not a card refund. Any unresolved holds remain separately visible until resolved. A payer can request an eligible cash refund from their original unspent purchase lots.

For tiny charges, allocate integer micro-units using deterministic largest-remainder allocation and a stable tie-breaker. Verify that allocation shares sum exactly to the charge. Per-payer rounding must never increase the table total.

5.5 Changes, departure, and disputes within the party

A payer may withdraw unreserved authorization at any time. Already sent work remains attached to its original reservation. If the payer leaves or is removed, pause further use of their remaining authorization by default. They may explicitly sponsor continued play after leaving.

A host can lower quality or pause without new consent. Raising the permitted rate above a payer's approved ceiling requires consent; otherwise exclude that payer from future higher-priced work and show the reduced funded runway. A new host does not inherit authority over cards or balances.

If one party member disputes a card charge, freeze the affected lot and its unspent allocations, not unrelated users' balances. The company bears consumed-service loss unless a legitimate recovery process applies. Never debit the host or other players to cover the disputed payer.

5.6 Auto-reload: later release only

Auto-reload is optional and off by default. Separate consent specifies the trigger, purchase amount, daily/monthly maximum, eligible session scope, payment method, and cancellation process. Saving a card is not consent to repeatedly charge it. Off-session payment failures or required authentication pause replenishment; they do not create negative credit. Stripe documents explicit-consent and future-use requirements. [S14]

No host can turn on auto-reload for another contributor. A payer opting into $20 reloads up to $60/month has a hard bank-charge authorization, independently of the session pot's exposure.

6. Authoritative usage billing and accounting

6.1 Keep five balances distinct

1. **Cash collected:** A processor or bank balance, reduced by fees, refunds, disputes, and payouts.
2. **Customer service-credit liability:** Purchased service not yet delivered; includes available and encumbered balances.
3. **Session allocation:** A subledger restriction on the customer's liability, not extra money.
4. **Retail reservation:** Maximum customer exposure for a specific admitted operation.
5. **Supplier liability:** What DungeonFlux may owe an API provider, including uncertain outcomes.

Retail and supplier reservations are related but not equal. A $0.60 maximum supplier request can have a $0.90 retail quote, plus a payer-specific tax treatment if required. A company-sponsored failed request can retain supplier liability while the customer's reservation is released.

Do not fund ordinary company expenses with unearned customer obligations. Keep sufficient business liquidity for unspent purchased credit, refundable services, pending provider exposure, processing losses, and disputes. This is a treasury policy, not a claim of statutory escrow or deposit insurance. Accounting treatment must be reviewed with the company's accountant.

6.2 Atomic admission and dispatch

A single paid request follows this sequence:

`Requested → Quoted → Reserved → DispatchPrepared → Sent → ResultKnown / OutcomeUnknown → Settled / CustomerReleased`

Before reserving, verify account status, membership, permissions, funding-lot status, consent version, session price version, amount, tax treatment, provider route, content rights, session owner fence, storage capacity, concurrency, and every spending limit.

Inside one database transaction, lock payer allocations in a stable order, validate idempotency, reserve retail amounts, reserve supplier exposure, and write an effect/outbox record. Commit before sending anything to a provider. The worker claims that exact effect under an attempt token and current owner fence. Persist `DispatchPrepared` before outbound network activity.

For every operation retain `operation_id`, `attempt_id`, provider idempotency key or job identifier where supported, customer quote ceiling, supplier maximum, funding-lot shares, price version, audience, canonical request digest, consent revision, and timestamps. Never log raw credentials or private prompt bodies to the money ledger.

No network protocol provides magical exactly-once execution across the database and every provider. Design for idempotent internal effects and explicitly uncertain external outcomes.

6.3 Settlement rules

Outcome	Customer treatment	Supplier accounting
Valid output delivered and accepted under contract	Settle known retail units, at or below quote; release surplus	Record actual or reconciled supplier charge
User canceled before dispatch	Release hold; no usage charge	Release supplier exposure
User canceled after dispatch	Follow the disclosed accepted-job policy; no more than quote	Keep exposure until outcome is known
Provider rejected before any billable work	No customer charge	Release only when confirmed safe
Technical failure, invalid output, duplicate, or platform defect	Release or credit customer charge under the proposed policy	Record company expense if billed
Provider accepted but result/status is unknown	Display an unresolved hold briefly; do not automatically resend	Keep worst-case supplier exposure until reconciliation
Speculative asset never used	No customer charge under pilot policy	Company bears waste within its own speculation budget
User explicitly requests an alternate valid output	New quote and operation	Separate supplier charge

Suggested customer experience: investigate unresolved retail holds immediately; after a disclosed short deadline such as 15 minutes, absorb the uncertainty and release the customer hold unless the user explicitly elects to wait. The supplier liability remains reserved on the company's ledger. A late result then becomes company-funded; do not surprise-charge the customer after releasing the hold. Finance reconciliation can continue for hours or days without keeping a player's balance hostage.

6.4 Money representation

Use one currency per funding lot/session; USD only initially. Represent internal calculations in integer micro-dollars or a checked exact decimal type; never binary floating point. Payment processor charges/refunds use integer cents. Store tariff multiplication as a rational or exact decimal, preserve remainders, and enforce nonnegative available funds in the database transaction.

At account closure or a cash refund, settle/refund whole cents according to a disclosed rounding rule that cannot benefit from repeated micro-refunds. Persist any sub-cent adjustment as an explicit rounding entry. Screens show cents while detailed exports retain the exact underlying usage units and aggregation.

6.5 Double-entry financial journal

Use an append-only journal with balanced entries per currency and immutable references to the external event. Correct mistakes with reversing entries, never by editing history.

Illustrative $20 purchase with no tax:

Event	Debit	Credit
Confirmed payment	Processor clearing $20.00	Customer service liability $20.00
Domestic processing fee	Processing expense $0.88	Processor clearing $0.88
Processor payout	Bank $19.12	Processor clearing $19.12
$1 service consumed	Customer service liability $1.00	Service revenue $1.00
$0.60 supplier charge	Cost of services $0.60	Vendor payable or prepaid supplier asset $0.60

Allocations and reservations are tracked as liability reclassifications or a rigorously reconciled encumbrance subledger. They are not revenue. Separate purchased credit, promotional credit, tax payable, refund payable, provider prepayments, and chargeback loss. Promotional credit has no cash refund value and cannot be moved into purchased credit.

The accountant decides formal recognition and presentation. The engineering invariant is that bank movement, customer obligations, consumed services, and supplier expenses can all be reconciled without inventing money.

6.6 Webhooks and retries

Store verified raw webhook payloads with controlled retention and limited access. Verify signatures against the unmodified body, reject invalid signatures, and put successful deliveries into a durable inbox before acknowledging. Process asynchronously with unique constraints and retryable failures. Return success quickly after durable receipt; do not hold the connection while performing long business workflows.

Out-of-order refund and dispute events must not resurrect credit. Reconcile by authoritative processor state and transaction identity, not by assuming event order. The order's payment intent, not a user-supplied email address or redirect parameter, binds it to the payer.

Outbox processing should use a lease and attempt generation; a delayed completion from an old attempt must not overwrite a replacement. This same rule applies to media resources and addresses the stale-completion race identified in the earlier source review.

6.7 Refunds, tax, and disputes

Users may request refunds of eligible unspent purchased credit. The interface previews the exact purchase lots, amount, released allocations, pending holds, and destination card suffix. Refund only to the original payment method; Stripe documents this restriction. [S13]

Suggested policy for a pilot: unused purchased service credit is refundable without an arbitrary expiration fee, subject to lawful exceptions and fraud review; consumed services are refundable for documented service defects or legal entitlement. Absorb ordinary refund-processing friction rather than obscuring it. Stripe generally does not return original processing fees, so model that cost. [S16]

A cash refund transaction locks the refundable lots, marks the amount unavailable, calls the processor with an idempotency key, and finalizes only after the definitive refund status. A failed refund returns to an unresolved/refund-review state, not immediately to both cash and credit.

A tax specialist must decide where service is sourced, whether funding or consumption is taxable, which product classification applies, and registration duties. Do not charge tax twice. Preserve payer billing location and tax treatment per lot; different contributors may have different tax treatment. The pot's displayed usable service balance and its maximum bank-charge amount must be unambiguous. Launch availability can be limited to reviewed jurisdictions until this works.

7. Billing governance and customer protection

7.1 Default protections

Never make the personal balance negative. Never spend one payer's unused funds on another campaign without authorization. Never increase a payer's individual maximum because the host increased the session maximum. Never send a paid request without a durable reservation. Never count a refunded or disputed funding lot as spendable. Never charge for replaying a completed generation.

Add provider account/day/month company caps independent of customer funding. Customers having credit does not imply the company has unlimited provider credit or risk capacity. An exhausted provider account opens a circuit breaker and changes the service's availability; it is not a reason for uncontrolled retries.

7.2 Customer statements

A bank receipt documents a purchase. A usage statement documents how service credit was consumed. A session statement summarizes multiple payers but does not become a tax invoice issued to the host for everyone else's purchases.

Each usage line contains a readable category, local time, session/campaign, quantity, price version, final retail charge, relevant reservation/reference, and status. Keep provider request IDs and internal error detail out of the ordinary customer screen but available in support diagnostics.

Examples: `Active table time — 42m 18s`, `NPC narration — 3.2 audio minutes`, `Scene image — 1 accepted image including references`, `Technical failure — credited`. A hidden prompt should not appear on another payer's statement simply because they helped fund the session.

7.3 Financial operations

Daily reconcile processor captures/refunds/disputes against ledger entries, processor payouts against clearing, and supplier usage against dispatch records. Investigate unexplained differences; do not use a balancing adjustment as a substitute for understanding a missing event.

Track company-paid waste, approved customer spend, unsettled provider liability, and customer-credit liability separately. Alert on rising retry cost, unexplained reserve age, duplicate settlement attempts, and low provider prepaid credit.

A manual adjustment requires a reason, linked support case, immutable actor ID, and a reversal path. Require a second approver above a chosen threshold before scaling beyond a solo-founder pilot. Support cannot alter a balance by SQL update.

8. Information architecture and page inventory

Use ordinary readable URLs and accessible responsive layouts. The existing Rust/WASM game remains the play surface. The marketing site can be lightweight HTML/CSS; account and billing pages should use the existing Rust web stack or server-rendered Rust templates rather than introducing an unrelated frontend framework solely for checkout.

Page	Route	Core purpose	Launch priority
Home	`/`	Explain experience and shared-cost model	Required
How it works	`/how-it-works`	Show invite, funding, play, save	Required
Pricing	`/pricing`	Explain platform fee, AI usage, caps, storage	Required
Demo	`/demo`	Reviewed recording or bounded sponsored demo	Required
Rules / supported content	`/rules`	Accurate subset, source attribution, limitations	Required
Help / status	`/help`, `/status`	Setup, billing, incidents	Required
Legal	`/legal/terms`, `/legal/privacy`, `/legal/refunds`, `/legal/credits`, `/legal/content`	Contract, privacy, financial and content policy	Required
Login / sign-up	`/login`, `/signup`	Hosted authentication entry	Required
Callback / verification	`/auth/callback`, `/verify-email`	Secure session creation and verified payer eligibility	Required
Recovery / security	`/account/recovery`, `/settings/security`	Recovery, methods, sessions, MFA	Required
Dashboard	`/app`	Resume campaigns, create, join, balance	Required
Campaign creation	`/campaigns/new`	Scope, party, rules, style, funding defaults	Required
Campaign overview	`/campaigns/:id`	Members, last save, sessions, assets	Required
Campaign settings	`/campaigns/:id/settings`	Access, content, host transfer, archive	Required
Invite / join	`/join/:token`, `/join`	Accept invitation or enter code	Required
Lobby	`/sessions/:id/lobby`	Seat selection, checks, funding, ready	Required
Player / shared display	`/sessions/:id/play`, `/display`	Actual play, privacy-filtered views	Required

Page	Route	Core purpose	Launch priority
Session summary	`/sessions/:id/summary`	Story recap, charges, saves, next session	Required
Character library	`/characters`, `/characters/:id`	Templates, portraits, campaign history	Required
Character editor / import	`/characters/new`, `/characters/import`	Build, upload, validate, quote generation	Required
Billing dashboard	`/billing`	Credit, reservations, purchases, receipts	Required
Add credit	`/billing/add`	Amount, tax, checkout handoff	Approval-gated
Payment return	`/billing/return`	Pending/success/failure status, no client crediting	Required
Session funding	`/sessions/:id/funding`	Pot, allocation, consent, rate policy	Required
Usage / receipt detail	`/billing/usage`, `/billing/transactions/:id`	Explain and audit charges	Required
Payment methods / refunds	`/billing/payment-methods`, `/billing/refunds`	Secure method management and eligible refunds	Required
Storage / assets	`/storage`, `/campaigns/:id/assets`	Quota, privacy, archive, export	Required
Saved sessions	`/campaigns/:id/saves`	Save history, resume, safe branch operations	Required
Account data	`/settings/data`	Export, deletion, consent	Required
Auto-reload / membership	`/billing/auto-reload`, `/membership`	Later optional recurring features	Deferred
Operator console	Separate protected admin surface	Sessions, finance, incidents, moderation, recovery	Required, minimal scope

9. Detailed page specifications

9.1 Home page

Primary goal: An organizer understands the product, payment model, and next step without needing an explanation of AI tokens.

Header: brand, How it works, Pricing, Supported rules, Help, Sign in, and one primary `Start an adventure` action. A secondary `Join a table` action must be easy to find for invited players.

Hero headline: **“Your party. A living world. A budget you control.”**

Hero body: “Play together on a shared screen, with every player joining from their own device. Share the cost, choose how much the table can spend, and return to the same characters and story next time.”

Primary CTA: `Create a free account`. Secondary: `Watch a real session`. Keep the recording honest: identify prerecorded generation, supported party size, and any prototype limitations. No fabricated testimonial, active-user count, or claim of complete rule coverage.

Below the hero, use these sections in order:

1. **The actual experience:** a real short gameplay clip showing the shared display and a player phone, with captioned audio.
2. **Four steps:** create a table, invite friends, agree on a budget, play and save.
3. **Shared budget example:** show three people allocating 20/10/$10, an $8 maximum hourly setting, and the distinction between available, reserved, and consumed funds.
4. **Persistent characters and stories:** show a saved character and campaign resume card; explicitly say an empty play balance does not delete progress.
5. **Pricing preview:** “Proposed pilot pricing: $2 per active table-hour plus metered AI usage. One table fee, not one per player.” Final public wording follows approved pricing.
6. **Control and privacy:** private player information, opt-in microphone, spending caps, free pause, and approved asset reuse.
7. **Current scope:** supported rules/adventures, device requirements, maximum tested players, and pilot status.
8. **FAQ:** does every player pay; what happens to unused contributions; what happens at zero balance; what is saved; what is billable; how refunds work.
9. **Final CTA and footer:** create, join, legal links, status, support, attribution, and business identity.

Do not load the full game WASM bundle just to render the homepage. Optimize accessibility, text readability, captions, keyboard access, and reduced-motion behavior. The financial explanation must not depend on an animation.

9.2 Pricing page

Show the table fee, AI markup basis, storage allowance, sample calculations, and premium-media approvals. Provide an estimator with duration, party size, mode, maximum spend, and preparation allowance. Party size changes the displayed per-person split, not the base platform rate; it can change actual AI consumption and must be labeled accordingly.

Display `estimated`, `maximum authorized`, and `actually charged` as different concepts. Explain pauses, provider delays, reused assets, failures, taxes, and refunds. Show the version/effective date of the price schedule. Include a `How billing works` expandable example and a link to detailed funding terms.

9.3 Login and account creation

Present `Continue with Google` and a supported email login method through the hosted identity provider. Existing users go straight to their dashboard or original invitation. New users complete display name, adult-age attestation, terms/privacy acceptance, and separate optional marketing consent. Require verified email before funding or hosting paid sessions.

Use a Rust server callback, authorization-code flow with PKCE where supported, state/nonce validation, strict redirect allowlists, server-side token storage, and secure HttpOnly session cookies. Do not store payment authority or long-lived auth tokens in localStorage. Do not automatically link accounts solely because an unverified provider returned the same email.

States: invalid/expired verification, already-used link, provider outage, wrong account on an invitation, canceled sign-in, suspended account, and return to the interrupted checkout. Recovery should not remove funding history or quietly create a second balance.

Use the hosted reset and MFA screens where possible. WorkOS's current hosted UI already includes core auth and recovery handling, but the product still owns authorization and application sessions. [S10–S11]

9.4 Dashboard and campaign overview

The dashboard prioritizes `Resume`, `Join`, and `Create`. Show campaign cards with last played, party, last durable save, next planned session, and owner. Personal available credit is visible only to the signed-in user. Add useful empty-state copy: “Your first adventure starts with a table. Invited already? Enter a room code.”

The campaign page has Overview, Sessions, Characters, Assets, Funding, and Settings tabs. `Resume` checks version compatibility, existing active owner, participant consent, and funding before starting paid work. An archived campaign can be viewed and exported without purchasing play time. An outstanding balance dispute must not erase the underlying story.

9.5 Create-campaign wizard

Step 1: name, adventure type, supported rules, content rating, and player count. Step 2: characters or later selection. Step 3: voice/image defaults and spending ceiling. Step 4: invitation permissions and funding model. Step 5: summary and create.

Creation itself is free. Any AI-generated campaign setup appears as a separate quote with a maximum and funding source. A failed setup cannot leave the customer with an unexplained charge or a missing campaign. A draft saves between steps.

9.6 Invite and lobby

An invite preview reveals only campaign name, host display name, scheduled time if present, and approved content-rating information. Do not expose private notes, assets, or finance before membership. Invalid, expired, full, revoked, and already-joined states need distinct messages.

Lobby cards show players, selected characters, readiness, audio/microphone permissions, and display pairing. A browser can join without paying when someone sponsors the table. Show `No charge until the host starts` except for explicitly approved setup jobs.

A short room code is a discovery/approval mechanism, not sufficient authority to reclaim someone else's paid identity. Seat recovery requires the same authenticated user or a deliberate host transfer. Pair a TV with an expiring one-time challenge; its token is read-only, cannot spend, and cannot reveal private player knowledge.

9.7 Billing dashboard

Show five clearly labeled amounts: available purchased credit, promotional credit, allocated to sessions, reserved for pending work, and consumed this period. Include Add credit, View usage, Receipts, Payment methods, and Request refund actions.

The transaction list differentiates purchases, allocation/release, usage, credits, refunds, disputes, and adjustments. Filters include campaign, date, category, status, and funding lot. An allocation must not look like a card charge. A refund pending must not look completed.

A payer can drill into a charge and see what happened, which quote was approved, which session used it, and how to report a problem. Keep original receipts and current statuses, not just a mutable latest balance.

9.8 Add-credit page and payment return

Amount controls are 10/20/$50 plus a bounded custom input. Display `You receive $20.00 of DungeonFlux service credit`, applicable tax, and exact total charged. Clarify nontransferability and eligible refund terms. A separate unchecked control can save the payment method; do not bundle it with auto-reload consent.

After Checkout, show pending status while the webhook completes. Poll a read-only purchase-status endpoint with backoff. Buttons allow returning to the campaign without creating a second purchase. Failed authentication lets the user retry the same purchase where safe, not automatically create duplicates.

9.9 Session funding page

Header: campaign/session, host, current price version, active platform fee, and selected ceiling. Main card: available pot, open holds, spent, estimated runway range, and end-of-session treatment.

Payer panel: own available credit, amount to allocate, own maximum, and approval checkbox summarizing what the host may trigger. Show whether other players can see the contribution amount. A change preview distinguishes new money from reassigned existing credit.

Host panel: maximum hourly spend, whole-session ceiling, optional generation toggles, per-job approval threshold, and split mode. Display each payer's consent status without revealing their card data. `Start` is disabled when essential startup work is not fundable or permissions/consent are incomplete.

9.10 In-game spend controls

On the host's device, use a discreet persistent budget chip: `Pot $24.80 • Held $0.70 • $8/hr maximum`. Expand for categories and remaining authorized exposure. On each player's device, show their own remaining authorization and a `Contribute` action. The TV can show low-funds status without exposing individual payer details.

At a threshold, explain the actual action: “Optional new pictures are paused. Add funding or continue with existing art.” At zero available credit, preserve the state, stop new paid requests, and show Resume when funded. Do not put an unexpected checkout overlay on every player's device or interrupt a completed paid cinematic merely because remaining credit is zero.

Premium quote: output type, maximum total price, who pays, expected latency range when known, cancellation policy, and `Generate for up to $X`. Never present a vague `Improve` button that spends money.

9.11 Session summary

Separate story and finance. Show completed story milestones, character progress, save timestamp, and next-session invitation. Finance shows active time, AI categories, technical credits, total consumed, unsettled holds, own share, and unused allocation released.

`Download recap` and `View save` reuse existing content. A new AI-written extended recap or highlight video is a separately quoted optional job. A session is not financially final while unknown reservations remain; label the statement provisional and notify payers when it finalizes.

9.12 Character library, files, and save screens

Character cards show portrait, name, build/rules version, campaign instances, last update, and visibility. Actions: view, duplicate template, edit, export, archive. Show the distinction between reusable template and campaign-specific progression; changing a template does not rewrite a live campaign's inventory or XP.

The import screen accepts reviewed structured formats and image uploads, validates limits, previews changes, and asks for confirmation. A portrait generation has its own quote. Private notes and draft portraits are private; shared campaign appearance is a separate audience designation.

The storage screen breaks down original uploads, approved generated media, videos, exports, and archived files. Explain what can be deleted safely. Deleting an asset required by a live save is blocked or replaces its presentation with a safe placeholder after confirmation; it never corrupts game state.

The saves screen displays chapter, durable revision, version, timestamp, participants, and compatibility. `Resume` uses the latest authoritative state. `Create a branch` is distinct from `Restore a backup`. Neither action rewinds the financial ledger or refunds previously delivered services.

9.13 Operator console

Minimum areas: active sessions and owners; payment inbox; purchases/refunds/disputes; ledger reconciliation; provider incidents and cost caps; storage jobs/quarantine; account safety; restore drills; and deployment version.

Admin sessions require MFA, short idle expiry, audited actions, and narrow roles. Support sees aggregate diagnostic data by default, not private roleplay text. An operator can disable a provider or pause new spending independently of game read-only access. No admin has a generic `set balance` button.

10. End-to-end user journeys

J01. First organizer creates and funds a table

Home → real demo → create account → verify email → create campaign → choose supported rules/content → select budget → buy service credit → webhook confirms → allocate to session → invite friends → lobby checks → start. Charges begin only under the displayed preparation or active-session rules. A payment interruption returns to the existing purchase, not a new one.

J02. Invited player joins without paying

Open invite → see host/campaign preview → authenticate or create account → accept membership → select/create supported character → allow or decline microphone → mark ready → play using sponsored pot. The player never has to enter a card to occupy a sponsored seat. Refusing microphone access leaves typed play available.

J03. Teammate contributes

Funding page → view rate and split → select own maximum → purchase credit if needed → approve allocation → join funding epoch. The contribution immediately changes available pot only after confirmed funds exist. The teammate can later withdraw unreserved authorization.

J04. Returning group resumes a saved campaign

Dashboard → campaign → last save details → resume lobby → validate content/runtime compatibility → reconnect each owner to their character → fresh session price/consent → fund → start. Rehydration uses saved rules results and paid assets without calling generation APIs. An old session's authorization does not silently fund a new one.

J05. Low balance during a session

Budget warning → optional work throttled → host chooses lower-cost mode, pause, or contribution request → one payer manually adds funding → confirmed payment/consent updates new work allowance → continue. No duplicate charge and no lost progress. An already-admitted job may finish within its old reservation.

J06. Host requests an expensive cinematic

Host opens quote → checks payer scope and remaining per-hour/session limits → payers approve additional exposure if needed → reserve → dispatch → show pending → deliver accepted asset → settle. Declining changes nothing. A quote expiry refreshes the price before approval.

J07. Player leaves or is kicked

Revoke seat and new private-media access → stop future use of remaining allocation by default → retain original shares for in-flight work → release unreserved credit → preserve the player's personal character template. Host cannot keep their unused contribution. Campaign history retains an appropriate pseudonymous record where necessary.

J08. Host disconnects or transfers control

Stop spend increases; maintain existing authorized play only within the disclosed continuity policy → allow delegated co-host to manage session → reconnect original host or explicitly transfer hosting → keep ownership, consent, and payment attribution separate. On total party disconnect, pause billing and checkpoint safely.

J09. Provider outage or uncertain job

Circuit breaker detects failure → stop new requests to affected provider → preserve supplier exposure on ambiguous calls → pause blocking platform time → use paid cached output or safe fallback → release/credit customer charge according to policy → notify status. Retrying an unknown job requires reconciliation or supported idempotency, not a new arbitrary request ID.

J10. Server crash and restart

Client reconnects with prior operation ID → new runtime claims a higher owner fence → loads database checkpoint → verifies money and provider dispatch state → returns prior committed result or resumes safe pending work → does not replay a payment or generation. Missing financial reconstruction keeps paid work frozen while read-only campaign access is restored.

J11. Upload a character portrait

Pick file → client shows size/type limits → server allocates quarantine key and upload ticket → private upload → finalize → verify bytes/checksum/dimensions → scan and sanitize → promote accepted asset → assign personal/campaign audience → update character revision. Failed validation shows a reason and removes temporary bytes without altering the saved character.

J12. End, settle, and play again

Host ends session → stop timer/new AI admission → finalize accepted actions and last save → settle known jobs → release unused allocations → show each payer's statement → schedule next session. Any new follow-up generation is a separate opt-in quote.

J13. Refund unused credit

Billing → eligible purchases → release unreserved session allocations → preview original-method refund → reauthenticate if required → create refund intent → processor confirms → show completion. A pending provider hold, tax adjustment, or dispute prevents only the affected amount from refunding.

J14. Export or close account

Settings → request export → reauthenticate → build audience-filtered package → short-lived private download → optional delete request → resolve unspent credit/refunds and campaign ownership → remove personal data according to disclosed retention → expire export. Required financial records remain under applicable retention policy, with access restricted. Do not delete the entire party's campaign because one member leaves.

11. Notifications and UX behavior

Send transactional notifications for email verification, new sign-in/security changes, invitations, payment confirmed, payment failed, contribution accepted/released, budget threshold, paused-for-funds, refund updates, session settlement, export ready, deletion schedule, and a material incident. Marketing communications require separate consent.

A budget warning is actionable and deduplicated. Notify the host and affected payers, not every spectator. Choose in-app alerts first, email for durable receipts and unresolved problems. Never put full payment instruments, private character secrets, provider credentials, or signed asset URLs in analytics events.

Every financial mutation has pending, success, retryable failure, final failure, and unknown-result states. Use a stable operation ID across retries. Disable duplicate submissions for usability, but enforce idempotency on the server because disabled buttons are not a correctness mechanism.

Accessibility requirements include keyboard navigation, visible focus, non-color-only statuses, captioned media, reduced motion, readable finance text, and screen-reader labels for live budget changes. Do not announce every microcharge through a screen reader; aggregate at useful intervals.

12. DigitalOcean hosting plan

12.1 Architecture



PostgreSQL is the authority for money and accepted game state. Spaces is the authority for finalized binary objects. A Droplet disk is disposable working storage, not the only copy of any character, campaign, or customer-paid asset.

The control plane handles authentication, memberships, billing, funding, session routing, webhooks, storage authorization, and operations. Campaign runtimes handle game authority and permitted views. Background workers handle provider calls, scanning, exports, and reconciliation through the same durable admission mechanisms.

Do not put Kubernetes, a separate message broker, a separate vector database, or a multi-region active/active financial system on the initial critical path. Use PostgreSQL jobs/outbox tables and bounded worker pools. DigitalOcean supports pgvector; validate extension availability and behavior on the selected managed edition rather than assuming a local PostgreSQL build transfers unchanged. [S19]

12.2 Region and network

Start in one US region where the chosen Droplets, managed PostgreSQL edition, and Spaces are jointly available; NYC3 is a candidate, not a claim about your existing account. Keep application, database, and media-origin region aligned. Benchmark end-user latency and provider RTT before final selection.

Create separate production and staging environments, database credentials, signing keys, payment modes, provider budgets, and buckets. Staging uses fake payments and low/no-cost fixtures by default. Do not give a preview branch production finance or provider credentials.

Use VPC networking for application/database communication, managed-database trusted sources, TLS verification, and cloud firewalls. Only the reverse proxy receives public gameplay/account traffic. The database, worker control interface, debug tooling, and admin services must not be publicly reachable.

HTTPS endpoints terminate through a carefully configured proxy or load balancer. The game uses WebSockets, so configure heartbeats, backpressure, connection drain, and reconnect independently of session ownership. DigitalOcean documents a special one-hour idle timeout for load-balanced WebSockets; this is not a promise of uninterrupted sessions through deployments. [S17]

12.3 Pilot resource bill

Published Basic Droplet rates include $24/month for 2 vCPU/4 GiB and $48/month for 4 vCPU/8 GiB. Spaces starts at $5/month. PostgreSQL plan and storage charges must be confirmed at checkout. [S1–S3]

Item	Pilot planning choice	Monthly amount
Application and bounded worker pool	1 × Basic 2 vCPU / 4 GiB	$24.00
Managed PostgreSQL	Budget a Standard 2 GiB plan	$30.00
Spaces subscription	250 GiB storage and 1 TiB outbound included	$5.00
Droplet daily backup, percentage plan	30% of $24	$7.20
Infrastructure subtotal	Before extra storage, email, monitoring, staging, taxes	**$66.20**
Operational allowance	Transactional email, alerts, extra backups	10–30
Pilot envelope	Not an HA service	**About 80–100/month**

Daily Droplet backup pricing is currently 30% of the Droplet price; weekly is 20%. These back up the VM, not a substitute for database and object recovery. [S5]

Do not promise a campaign count from this configuration. Qualify one, then several concurrent campaigns with real memory and CPU measurements. Separate workers onto another $24 Droplet if media processing or game memory interferes with webhooks and billing. The second Droplet with daily backup adds $31.20/month at these list prices.

12.4 Important managed-database plan change

DigitalOcean's account eligibility is not simply whether an account exists. The new restrictions began October 15, 2026 for accounts that had never created a PostgreSQL or MySQL cluster, and begin November 30, 2026 for other accounts. New Standard clusters are limited to selected small plans and cannot add standby or read-only nodes. Existing clusters are treated separately under the published grandfathering policy. [S4]

Having a DigitalOcean account does not prove that you can create the older low-cost Standard HA topology. Verify your account's actual cluster history and available plans before purchasing. Do not rush-create a database merely to rely on a presumed exemption.

For a conservative new-production HA budget, use Advanced Edition: the documentation lists an $130/month 8 GiB/2 vCPU primary, with matching standby billed separately. Thus primary plus one standby begins around $260/month before additional storage. [S2]

Item	Conservative paid-production baseline	Monthly amount
Application/worker nodes	2 × 4 GiB Basic	$48.00
Regional HTTP load balancer	1 node	$12.00
PostgreSQL Advanced primary + standby	2 × $130	$260.00
Spaces	Base subscription	$5.00
Daily node backups	30% × $48	$14.40

Item	Conservative paid-production baseline	Monthly amount
Subtotal	Before operational allowances	**$339.40**
Email, monitoring, extra backups	Planning allowance	20–50
Working envelope	Before staging and exceptional usage	**About 360–400/month**

A regional HTTP load balancer starts at $12/month. [S18] This topology improves node resilience; it is not automatically a multi-region disaster-recovery system or an availability guarantee. A controlled paid pilot may use a single database with explicitly limited service expectations, tested restores, strict exposure limits, and no uptime SLA. Broad paid launch should justify its resilience choice explicitly.

12.5 Runtime isolation and scaling

The current code includes process-global configuration and local runtime assumptions. The safer first hosted shape is one isolated native process/container per active campaign, with multiple bounded campaign processes placed on a worker Droplet. Do not allocate a VM per player or assume multi-campaign safety inside one process before auditing globals, caches, and tenant scopes.

A small trusted runner starts only signed known images with a read-only root filesystem, a campaign identity, narrow credentials, resource limits, and an expiring lease. It must not expose an unrestricted Docker socket to user-facing application code. The directory maps each campaign to its current worker and monotonically increasing owner fence.

For load-balanced traffic, an edge instance looks up the campaign owner and routes accordingly. Load-balancer stickiness alone is not ownership and cannot prevent split-brain decisions. During migration, stop new admission, drain/record outstanding work, checkpoint, transfer ownership, and reconnect clients. A failed old worker cannot commit under an obsolete fence.

Capacity formula for scheduling:

```
safe table slots = minimum(memory-limited slots, measured CPU slots, connection slots, database slots, provider concurrency slots)
```

For memory, subtract OS/control-plane headroom and reserve safety margin before dividing by measured p95 runtime memory. For CPU, measure burst behavior and blocking image tasks, not just mean owner-thread usage. If a runtime requires 1 GiB, a 4 GiB host cannot safely claim dozens of tables; if it requires much less, demonstrate that before increasing density.

Scale the worker pool when sustained memory/CPU, admission queue delay, or supported-slot usage crosses an established threshold; an initial target is at most 60–70% steady-state resource utilization with burst headroom. Drain before removing nodes. Powering off a Droplet does not necessarily end resource billing; destroy unneeded ephemeral capacity only after state is safely externalized. [S20]

12.6 Deployment and CI

Build Linux artifacts in CI, not on the production machine. Pin the Rust toolchain, dependencies, migrations, content version, and application build. Publish immutable container digests. Check native compilation, WASM compilation, linting, relevant deterministic tests, payment/idempotency contracts, and browser flows.

The current README reports a known crash in the full `df-tools` test suite. That must become a tracked blocker with an explicit reliable test matrix; do not hide it behind an indefinite filtered-only release gate. [R1]

Use expand/contract database migrations, compatible readers during rolling deployments, and a migration rehearsal against a production-shaped staging dataset. Rollback the binary only when the new schema remains compatible. Irreversible migrations require backup/restore rehearsal and an outage plan.

Deploy to a canary campaign before broad admission. Roll back on ledger mismatch, state loss, provider spend duplication, severe latency regression, or cross-tenant access. A feature flag can stop generation categories without destroying sessions.

12.7 Secrets and administrative access

Separate deployment, runtime, payments, storage, and backup credentials. Restrict runtime database roles; no superuser, database owner, or bypass-row-security role. Preserve tenant filtering in every query and use row-level security as defense in depth where appropriate.

Use a protected deployment secret store and short-lived CI credentials where supported. Never place production `.env` files in an image, repository, artifact bundle, browser response, or log. Administrators use individual identities, MFA, and a VPN/bastion or equivalent private path. Audit key rotation and rehearse recovery after a compromised provider key.

13. File storage, player characters, and saved campaigns

13.1 What belongs where

Information	Authoritative storage	Secondary copy / delivery
User identity linkage and preferences	PostgreSQL	Restricted account export
Character template and build selections	PostgreSQL, versioned rows	JSON/PDF export in private Spaces
Campaign character progression and inventory	PostgreSQL, committed game events	Checkpoint bundle in Spaces
Character portraits and original uploads	Private Spaces	Sanitized variants, authorized delivery
Campaign state, accepted decisions, dice, journal	PostgreSQL	Immutable compressed checkpoint bundles
Story memory and audience labels	PostgreSQL full-text/pgvector	Backup only; no universal public embedding index
Scene images, music, narration, cinematics	Spaces objects plus PostgreSQL asset metadata	Short-lived authorized URLs or gateway
Receipts and financial journal	PostgreSQL + processor records	Independent encrypted recovery journal/export
Temporary uploads and generation intermediates	Quarantine/temporary Spaces prefixes	Expire after bounded processing
Application static assets	Public marketing/static bucket or web deployment	CDN with versioned immutable paths

Information	Authoritative storage	Secondary copy / delivery
Account/campaign export archives	Private Spaces	Expiring download after reauthentication
Runtime working files	Ephemeral Droplet/container disk	Never the sole durable copy

A saved session is not merely a ZIP file on a VM. It is an authoritative database revision plus immutable references to its required media and versioned rules/content. PostgreSQL transactions preserve the relationship between accepted decisions and pending effects; object-store writes are coordinated but are not magically part of the same transaction.

13.2 Bucket layout

Create separate buckets or strictly separated permission domains for `public-static`, `private-media`, `uploads-quarantine`, `exports`, and `backups`. Use independent environments and narrow service keys. Object names contain opaque identifiers rather than email addresses, real names, or private story titles.

Example object key layout:

```
prod/users/<user-id>/characters/<character-id>/<revision>/<asset-id>.webp
prod/campaigns/<campaign-id>/assets/<asset-id>/<variant>.webp
prod/campaigns/<campaign-id>/checkpoints/<epoch>-<revision>.bundle
prod/exports/<user-id>/<export-id>.zip
prod/quarantine/<upload-id>/source
prod/backups/<dataset>/<date>/<backup-id>
```

These prefixes are organizational conventions, not authorization. Database ownership/audience records and restricted signing credentials enforce access. Shared visual assets remain private to the relevant campaign even if all its players can see them. The marketing site uses only explicitly licensed, public examples.

13.3 Asset metadata and finalization

Each asset row records owner scope, campaign/character, audience, asset type, MIME type, byte length, dimensions/duration, checksum, object key/version, provider lineage, licensing/consent flags, price/job identity, accepted status, retention class, and deletion state.

Lifecycle:

`Uploading → Quarantined → Validated → Ready → Referenced / Unreferenced → Archived → Deleted`

Write immutable finalized objects under unique keys. Verify integrity and existence before committing a reference that promises delivery. A provider output is not customer-delivered merely because the provider returned a URL; it must be copied into controlled durable storage, validated, and associated with the correct audience. Use retryable finalization and orphan cleanup to resolve object/database gaps without duplicate generation.

Do not treat a temporary provider CDN URL as permanent storage. Do not rely on a 512-entry in-memory serving registry to decide whether a paid asset exists. The durable catalog is authoritative; caches may evict bytes without invalidating ownership or producing permanent broken references.

13.4 Upload security

Initial formats: JPEG, PNG, WebP for images; one versioned JSON schema for character imports. Defer arbitrary HTML/SVG, executables, large archives, and unstructured PDF ingestion until a sandboxed ingestion path exists.

Suggested starting bounds: 10 MiB per uploaded image, 20 megapixels decoded, 1 MiB structured character JSON, and explicit per-user in-flight upload and storage quotas. Validate actual bytes and decoded structure, not filename extensions. Remove EXIF/location metadata, bound decompression, reject path traversal and remote references, and scan files before use.

Issue upload permission only after authentication, quota reservation, and object-key allocation. Use a short-lived one-object upload ticket; where the storage API cannot enforce all desired restrictions at upload, quarantine and verify before promotion. Complete via a finalize endpoint that checks expected size/checksum and ownership. Expire incomplete multipart uploads and abandoned objects.

User-provided text and images are untrusted game content. They cannot alter model system policy, authorize spending, choose arbitrary provider URLs, or become executable server instructions.

13.5 Private delivery and caching

Authorize against current membership and asset audience, then issue a short-lived presigned origin URL, typically about five minutes, or stream through the authorized gateway for stricter access requirements. Previously issued URLs remain usable until expiry, so immediate revocation requires gateway enforcement or a different access design. Never claim instantaneous revocation while handing out bearer links.

DigitalOcean documents private objects and time-limited presigned access. It also states that presigned files do not get normal Spaces CDN caching and can incur double bandwidth costs if routed incorrectly through the CDN. Therefore, use the CDN for public static assets; use private origin delivery initially for player/campaign media. [S12]

Use `Referrer-Policy: no-referrer`, redact signed URLs from logs/analytics, and avoid public cache headers for restricted files. Streaming video seeks and range requests must preserve authorization. Do not expose backup or export buckets through the public website.

Spaces and managed databases document encryption at rest; that is not the same as application-level isolation. Encrypt particularly sensitive export/backup packages with keys outside the bucket and protect key recovery separately. Droplet local disks have a different security model, so avoid durable private caches there and encrypt sensitive temporary files when necessary. [S21–S23]

13.6 Character persistence model

Store reusable identity, public appearance, private backstory, supported build choices, provenance, and revision history in the personal template. A campaign creates its own character instance, bound to a rules/content version and owning account. Equipment, XP, currency, injuries, relationships, and campaign-specific abilities belong to that instance.

Exporting or duplicating a template does not duplicate valuable inventory in a live campaign. An imported sheet is a proposal validated by the rules engine, not an authoritative replacement of server state. Use optimistic concurrency checks so two tabs cannot silently overwrite the same character revision.

A player's personal library survives leaving a campaign. The campaign retains only the data it legitimately needs for its history and authorized continued play. Draft portraits and notes are private; publishing a portrait to the campaign is an explicit separate audience transition.

13.7 Save and resume contract

Commit every accepted game decision durably before acknowledging success. Periodically create compact checkpoints—for example at scene boundaries or every bounded number of decisions—and retain the events needed for replay. The exact interval is a performance decision measured against state size and recovery time, not a license to lose acknowledged actions.

Each checkpoint records schema, runtime/rules/content versions, campaign/session identity, owner epoch, last event sequence, RNG/dice provenance, pending effects, state digest, and asset manifest. Avoid indefinitely embedding every historical event in one ever-growing bounded snapshot. Compaction must preserve semantic and billing links and be validated against old saves.

The UI says `Saved` only after the authoritative database commit. A separate `Backup copied` status can describe asynchronous object backup. These are different guarantees.

Resume authenticates membership, claims a valid owner fence, checks version compatibility, reconstructs pending work, and restores client views. Access-token expiration cannot make the saved campaign unrecoverable. A safe migration either succeeds with evidence or leaves the old save intact and read-only.

Branching or restoring game state never restores old financial balances, reopens consumed authorizations, or resends already-paid provider work. Financial state has its own monotonic journal and remains forward-only. Replay uses recorded results and stored media. If an optional old narration file has legitimately expired, replay can omit audio; it must not silently buy another generation.

13.8 Proposed storage allowances

Data class	Initial policy	Charging behavior
Personal character library	Up to 10 templates and 100 MiB of personal uploads per verified account	Included; prevent account-farm abuse
Standard campaign media	2 GiB logical retained media per funded campaign	Included in pilot, with explicit quota display
Character and campaign progress	Reasonable bounded structured state, separate from optional media quota	Never stop essential saving because the picture quota is full
Large cinematics	Disabled by default; quote output and retained size	Generation quoted; storage expansion separate if needed
Export packages	Short-lived download, e.g. 7 days	Ordinary export included
Rejected/intermediate output	Delete after 24–72 hours, unless needed for a short support investigation	Company expense; not permanent customer storage
Temporary voice input	Do not retain by default after processing	Opt-in diagnostic recording separately governed
Inactive campaign media	Clear archive policy; no deletion solely because balance reaches zero	No silent overnight debit from the pot

These are initial product limits, not a claim of unlimited free storage. Preserve progress and approved essential media while the account/campaign remains within the disclosed retention policy. For long inactivity or product shutdown, give notice, a meaningful export window, and a documented process. Do not promise perpetual hosting.

At a media quota, stop new optional generation/uploads and let the user archive/delete eligible media or approve an explicit storage plan. Do not delete a referenced asset to make room without consent. A future archive subscription is a separate opt-in recurring charge, not an implicit use of play credit.

13.9 Storage cost model

Spaces currently includes 250 GiB stored and 1,024 GiB monthly outbound in its $5 subscription; additional storage is $0.02/GiB-month and transfer $0.01/GiB. [S3]

Stored amount, ignoring transfer overage	Illustrative monthly Spaces storage bill
250 GiB	$5
1,000 GiB	$20
5,000 GiB	$100

The bill applies to all actual stored bytes, including versions, duplicates, thumbnails, backups, and exports—not just logical customer quotas. A campaign's three GiB of media is inexpensive to retain, but repeated delivery to six devices can make transfer more important. Example: 0.25 GiB of newly delivered assets per viewer × six viewers × 1,000 sessions is 1,500 GiB before cache benefits, or about $4.76 of transfer overage at the stated included allowance. This is a bandwidth illustration, not a measured game profile.

Maintain object-size and download-byte telemetry per campaign. Keep shared media generated once per table where possible, while preserving private-content boundaries. Cross-campaign deduplication must not expose another customer's data or licensing.

13.10 Backups and disaster recovery

Managed PostgreSQL provides daily backups and point-in-time recovery within the documented seven-day window. Add independent encrypted logical exports and tested restores; a VM snapshot alone is inadequate. [S15]

Proposed retention: managed seven-day PITR; independent daily backups for 30 days; periodic longer-lived backups according to legal/data-retention review; versioned important objects with bounded lifecycle. Maintain at least one backup copy outside the production credentials' destructive authority. Versioning is useful but is not proof of immutable storage or a complete disaster-recovery system. Validate Spaces' supported S3 features rather than assuming full S3 compatibility. [S24]

Set recovery **targets**, then measure: routine session runtime restart under two minutes; service restoration from a tested regional backup within an hour; no acknowledged financial obligation silently lost or duplicated. Do not publish those as SLAs until demonstrated.

An older database restore can resurrect spent credit and forget a dispatched API call. Before paid admissions resume, reconcile processor transactions, the independent financial/dispatch journal, deletion/revocation records, and provider job status. If ambiguity remains, freeze affected financial operations. A snapshot's age is not permission to forget supplier liability.

Protect an append-only financial/dispatch recovery journal with independent credentials and high-watermark verification. Do not describe a normal Spaces bucket as WORM storage unless that exact property is verified. If true immutability is required, use a validated retention-locked service or independent control boundary; keep this separate from ordinary media-serving keys.

Rehearse restoring a database, a deleted asset, an expired session credential, and a partially dispatched provider operation. Record recovery time, missing events, reconciliation results, and who approved reopening spending.

13.11 Export and deletion

An account export includes that user's profile, character templates, permitted campaign excerpts, personal usage statements, and owned uploads. It excludes other users' private notes, hidden game-master state, credentials, and operator-only audit material. A host campaign export is likewise audience-filtered unless a separately authorized full backup is explicitly intended.

Require reauthentication for large/private exports, issue a short-lived download, record access, and expire the archive. Deletion removes live access immediately where feasible, removes unneeded primary objects within the stated policy, and ages encrypted backups out on their retention schedule. Preserve required financial records in a restricted store with a documented lawful basis; do not promise unconditional instant erasure of every backup.

Reference-count assets before physical deletion. A shared asset used by two permitted campaign branches must not disappear when one branch is archived. Use tombstones and a garbage-collection delay so failed transactions or delayed references cannot produce broken saves.

14. Security, privacy, content, and commercial readiness

14.1 Identity and session boundaries

Authentication proves who a person is. Campaign membership determines what they can see/do. Funding consent determines what may be spent. Keep those checks separate and re-evaluate them at the relevant boundary.

Validate WebSocket origin and authenticated scope. Use short-lived connection tickets bound to account, campaign, role, and intended device; rotate on revocation or host transfer. A display ticket cannot invoke player actions or financial commands. Rate-limit account creation, invitations, room-code guessing, uploads, generation requests, checkout creation, and recovery attempts.

Protect ordinary mutations against CSRF, enforce strict CORS, and verify JWT/session issuer/audience/expiry. Never trust a client-provided role, account balance, provider cost, content approval, or campaign owner. Authorization must also cover file downloads and exports, not just screen rendering.

14.2 AI-specific safety and spending integrity

Treat player prompts, imported characters, PDF text if later supported, NPC statements, and generated artifacts as data. They cannot authorize money or override provider policy. The deterministic game/commerce boundary decides legal actions, limits, and chargeability.

Use server allowlists for outbound provider hosts, explicit size/time limits, and no arbitrary remote-file fetching from user URLs. Prevent request chaining from becoming unlimited tool execution. A malicious roleplay instruction such as “generate 1,000 videos” must not bypass host settings or contributor consent.

Content controls should be consistent across text, pictures, voice, and video. Begin with a content profile acceptable to selected processors and suppliers. Do not make explicit sexual content, unauthorized real-person voice cloning, or protected likeness generation a launch feature. Record contributor permissions for uploaded media and lawful usage of any third-party content.

14.3 Business and rights checklist

Before paid launch: confirm the seller entity, business bank account, statement descriptor, support address, refund process, tax classification, payment approval, and insurance/counsel needs. Publish terms, privacy policy, funding/credit terms, refund policy, storage/retention policy, acceptable-use/content policy, and the supported-rules disclosure.

The official SRD/creator materials describe the licensed content scope. The SRD is not a blanket license to every D&D book, setting, trademark, or artwork. Keep exact attribution, source/content versions, third-party notices, and supplier-use permissions. Review the product/domain branding separately from rules licensing. [S25]

The README currently says the repository lacks a LICENSE file despite the manifest declaration. Resolve repository and third-party licensing before treating distribution rights as settled. [R1]

14.4 Privacy defaults

Microphone capture is opt-in; typed input works without it. Avoid retaining raw microphone audio by default. Disclose which providers process text, audio, images, and memory. Do not claim end-to-end encryption when the server and AI providers must read the content.

Private notes and secrets have audience labels enforced before retrieval, ranking, prompting, projection, and delivery. A shared pot does not make all funded content public to all payers. Deletion, revocation, and data export are product workflows, not support-only database scripts.

Launch as an adult service unless a reviewed minor-user flow exists. An age checkbox alone is not a comprehensive compliance program. Define abuse reporting, account suspension, appeals, content takedown, and law-enforcement escalation procedures with counsel.

15. Domain model and engineering contracts

15.1 Core persistence entities

Entity	Required properties and invariants
`User` / `IdentityLink`	Stable internal UUID; unique provider+subject; verified status; suspended/deleting state.
`Campaign` / `Membership`	Owner, role, invited/active/revoked, audience permissions, content/rules versions.
`Session`	State, price version, caps, funding epoch, active-meter segments, runtime owner fence.
`CharacterTemplate` / `CampaignCharacter`	Personal identity separate from campaign progression; optimistic revision; provenance.

Entity	Required properties and invariants
`PriceVersion`	Effective date, currency, platform rate, provider tariff, markup, tax policy version, supported products.
`PurchaseOrder` / `PaymentAttempt`	Amount, payer, processor IDs, idempotency, state, bill-to and tax evidence.
`FundingLot`	Original purchase, purchased/promo class, consumed/refunded/reserved totals, availability state.
`Allocation` / `Consent`	Session+payer, amount, remaining, approved rate/settings, version, withdrawal state.
`Quote` / `Reservation`	Exact retail maximum, supplier maximum, expiration, source allocations, immutable request digest.
`ProviderAttempt`	Stable attempt token, supplier job ID, dispatch status, outcome/uncertainty, reconcile evidence.
`UsageEvent`	Delivered quantity, time/category, quote and price version, funding shares, settlement status.
`JournalEntry` / `JournalLine`	Immutable double-entry postings; balanced by currency; external-event uniqueness.
`Refund` / `Dispute`	Original funding sources, processor state, tax reversals, frozen amounts, case reference.
`Asset` / `Upload`	Object metadata, owner/audience, scan state, digest, references, retention/deletion state.
`Checkpoint` / `GameEvent`	Durable revision, engine/content/rules version, hash, pending effects, replay links.
`WebhookInbox` / `OutboxJob`	Deduplication, lease, attempts, retry policy, terminal/unknown state.
`RuntimeOwner`	Campaign, worker, lease, monotonically increasing fence; no competing authority.
`AuditEvent`	Actor, scope, command, result, correlation, reason; protected retention.

Do not implement a second mutable balance authority in Redis or browser state. Derived balance caches must be disposable and reconcilable to the journal. Persistent financial uniqueness must include the business effect, not just the transport event ID.

15.2 Proposed service operations

These are semantic contracts, not permission to create dozens of new microservices. Expose them through the existing API conventions and typed protobuf/HTTP boundaries as appropriate.

Operation	Important preconditions	Result
Create purchase	Verified payer, approved product, amount bounds	Checkout reference and order ID
Get purchase status	Same payer or privileged audited operator	Authoritative pending/final state
Allocate funding	Spendable source lots, explicit consent, current session version	Allocation ID and exact remaining amount
Withdraw allocation	Payer ownership, unreserved amount	Released amount; holds unchanged
Update session policy	Host authority; payer-consent compatibility	New policy version or consent-required result

Operation	Important preconditions	Result
Quote generation	Authorized audience, permitted feature, pricing/current provider capability	Retail/supplier maxima and quote ID
Admit paid effect	Valid quote, all caps/funds/consents, owner fence	Durable reservation and outbox job
Settle usage	Exact attempt/result identity and proof	One charge and reservation release
Request refund	Eligible original sources, reauthentication/risk checks	Refund intent with asynchronous status
Start/pause/end session	Host/delegate authority and valid state transition	New session state and meter boundary
Claim/recover runtime	Lease and fencing rules	New owner epoch; prior owner fenced
Request upload / finalize	Owner, quota, type/size constraints	Quarantine ticket / accepted asset
Authorize asset	Current audience access and deletion state	Short-lived origin URL or authorized stream
Save character	Ownership, legal build, expected revision	New immutable revision
Export data	Reauthenticated owner and audience	Async export job, expiring access
Archive/delete asset	References, retention, permissions	Tombstone or explicit refusal
Branch campaign	Approved checkpoint, new identity, no money rollback	New branch with shared permitted assets

All mutation requests carry an idempotency key. Reusing the same key with a different semantic payload returns a conflict. A timeout returns an operation reference and lookup path so the client does not guess whether money or state changed.

15.3 Reuse the existing crate boundaries

Use `df-commerce` for pure money, consent, quote, and exposure policies; `df-persistence` for transactions, journals, scopes, and recovery; `df-auth` for trusted identity/capabilities; `df-session` for authoritative session decisions; `df-provider-api` and providers for supplier requests; `df-assets` / `df-media` for asset lifecycle; and `df-web` /player/display/UI for permitted presentation.

Introduce a real production composition root without rewriting the game engine. The current `df-tools` runtime can be a migration source, but production admission cannot depend on developer override flags, hard-coded credentials, or local-room reclaim behavior.

Add contributor allocations as a carefully modeled extension to existing commerce scopes. Do not replace the current supplier-exposure design with a simplistic “subtract from pot” helper. Preserve separate company supplier limits, customer retail quotes, and per-payer consent.

15.4 Mandatory behavioral invariants

- Available funds cannot become negative under simultaneous requests.
- Every settled customer charge maps to one approved quote or active-time policy.
- Every sent supplier attempt maps to one durable exposure record.

- Every monetary journal transaction balances in its currency.
- A stale job completion cannot settle a newer generation of the same resource.
- A revoked member cannot obtain new private asset access.
- Every acknowledged game decision survives ordinary process restart.
- A save branch or restore cannot replay payment, refund, or supplier dispatch.
- Failure of telemetry does not erase game state, while failure of durable financial admission blocks new paid work.
- A customer-visible “saved” or “paid” message corresponds to a durable authoritative outcome.

16. Observability, performance, and operations

16.1 Instrument the complete journey

Use one correlation chain from player input to command admission, provider quote/reservation, supplier response, accepted game decision, view publication, audio/image delivery, and settlement. Track server queue time separately from execution time. Retain model/provider IDs and price versions without logging private prompt contents by default.

Recommended initial service targets, to be qualified: p95 non-provider command acknowledgement below 250 ms within the selected region; immediate visible acknowledgement of model-backed input; no silently stalled view stream; bounded reconnect; ordinary dialogue materially below the currently reported long waits; and zero known financial invariant violations. These are release targets, not claims of current performance.

The server concurrency research indicates sequential model work can dominate latency. Optimize measured critical paths and overly broad locks before buying many CPUs. Do not infer aggregate campaign capacity from one thread's low average utilization. [R4]

16.2 Metrics dashboard

Business: activated groups, complete paid sessions, weekly returning groups, referred hosts, consumed revenue, outstanding credit liability, average revenue/table-hour, gross contribution, support minutes, refund/dispute rate, and acquisition cost.

Billing: purchases pending too long, duplicate webhook/business-effect attempts, reservation age, unresolved supplier exposure, reconciliation differences, price-version mismatches, cap refusals, and manual adjustments.

Gameplay: useful play versus blocked time, first action success, dialogue latency distributions, reconnect count, server-induced pauses, session completion, and durable-save lag.

Storage: retained bytes by class, upload rejection, quarantine age, broken asset references, expired export cleanup, backup age, restore success, and per-session transfer.

Infrastructure: worker slots, per-runtime memory, queue depth/age, PostgreSQL pool saturation, commit latency, disk pressure, object-store errors, and external provider error/credit status.

16.3 Alert and incident priorities

P0: money duplication, cross-tenant disclosure, incorrect funding ownership, leaked secret, or corrupted accepted state. Immediately stop affected paid admission; preserve evidence and invoke the incident runbook.

P1: widespread failed joins, inability to resume, sustained provider outage, stuck sessions, failed backups, or unknown exposure growing beyond cap. Degrade optional media, pause blocked billing, and communicate status.

P2: isolated picture quality, noncritical UI defects, slow optional exports, or recoverable individual upload failures. Track and remediate without unnecessarily ending all sessions.

A shallow health endpoint returning the last cached checkpoint is not proof that the session owner is healthy. Liveness, readiness, database health, provider availability, and gameplay-owner progress are separate signals.

16.4 Core runbooks

Provider outage: open circuit, stop new requests, preserve pending jobs, use safe cached output, pause blocked time, release customer holds under policy, reconcile invoices, then reopen gradually.

Webhook backlog: keep purchases pending, never mint provisional spendable credit, replay durable inbox, reconcile processor list, and monitor duplicate posting checks.

Ledger discrepancy: freeze affected money paths, snapshot evidence, compare journals and external references, repair with reversing/corrective entries, obtain review, and notify impacted users.

Database failure: stop financial/game writes, maintain status/read-only behavior where safe, fail over or restore, reconcile irreversible external effects, and only then reopen paid work.

Storage deletion or corruption: deny missing output rather than serving a wrong asset, restore by digest/version, repair catalog links, and provide a customer credit when purchased output cannot be delivered.

Secret compromise: revoke keys, disable provider spending, rotate application and storage credentials, inspect dispatch/audit records, and assess notification obligations.

17. Acceptance test catalog

The following tests are release gates, not optional code-coverage suggestions. Use fake providers and controlled clocks for routine CI; use deliberately authorized small live tests for external contract qualification.

ID	Scenario	Required outcome
PAY-01	Checkout redirect forged or replayed	No credit without authoritative payment success
PAY-02	Same webhook delivered repeatedly	One business posting only
PAY-03	Refund event precedes delayed success event	No resurrected credit
PAY-04	Payment succeeds after browser closes	Funds appear once on return

ID	Scenario	Required outcome
PAY-05	Two concurrent purchases/allocations	Correct source attribution and no lost updates
PAY-06	One card supports multiple payers/accounts	Risk review does not expose unrelated data
POT-01	Six concurrent requests exhaust a small pot	No negative balance or excess dispatch
POT-02	Payer withdraws while a job is admitted	Only unreserved funds released
POT-03	Host raises ceiling beyond consent	Incompatible payer is not charged without approval
POT-04	Contributor removed mid-job	Original hold settles correctly; new work stops using their funds
POT-05	Rate or split changes mid-session	New epoch affects future operations only
POT-06	Tiny proportional allocations	Exact table total; no systematic rounding leakage
JOB-01	Timeout after supplier may have accepted	Unknown exposure retained; no blind resend
JOB-02	Old completion after invalidation/replacement	Stale attempt rejected
JOB-03	Provider quota exhausted	Circuit breaker; actionable notice; no retry storm
JOB-04	Invalid or duplicate output	Correct no-charge/credit policy
JOB-05	Late result after customer hold released	No surprise retroactive charge
TIME-01	User pauses or everyone disconnects	Meter follows documented useful-service policy
TIME-02	Long blocking supplier wait	Platform billing paused for blocked interval
TIME-03	Clock skew, restart, repeated meter tick	No double charge or negative duration
REF-01	Partial refund spans several funding lots	Correct original sources, tax, and processor references
REF-02	Refund call times out	No double refund or premature reusable credit
REF-03	One contributor disputes after usage	No charge shifted to another contributor
AUTH-01	Room code guessed or old invitation reused	No takeover of authenticated paid seat
AUTH-02	Display token invokes payment or secret view	Denied
AUTH-03	Another member requests draft portrait	Denied; bearer grants expire according to policy
SAVE-01	Restart more than an hour after session end	Campaign resumes despite old credential expiry
SAVE-02	Crash after state commit before response	Retried operation returns original result
SAVE-03	Older save branch	Financial journal and paid generations remain forward-only
SAVE-04	Large six-player campaign	Checkpoint compaction and recovery stay within bounds
SAVE-05	Old schema/content version	Tested migration or safe read-only refusal
FILE-01	Oversize/decompression-bomb/wrong-MIME upload	Rejected safely before promotion

ID	Scenario	Required outcome
FILE-02	Signed upload to wrong object key	Denied or quarantined, never accepted as another user's asset
FILE-03	Revisit more than 512 prior assets	Durable catalog serves owned assets correctly
FILE-04	Large provenance manifest and restart	No loss of registrations or asset ownership
FILE-05	Storage quota exhausted mid-play	Essential save succeeds; optional media refused
FILE-06	Delete asset referenced by a save	No broken state; explicit reference handling
FILE-07	Revoked membership and existing signed URL	Documented expiry behavior; no new URL issued
FILE-08	Account export includes other-player secrets	Test must fail until filtering is correct
OPS-01	Worker drain during an active session	Controlled checkpoint/reconnect without duplicate owner
OPS-02	Database restore before recent payment/ dispatch	Reconciliation blocks unsafe reopening
OPS-03	Independent backup restore	Integrity, secrets, versions, and recovery time verified
OPS-04	Full finance reconciliation	No unexplained imbalance
UX-01	Sponsored player joins without a card	Complete supported session
UX-02	Payment pending/failed/succeeded states	Accurate wording and safe retry behavior
UX-03	Keyboard and screen reader billing flow	All controls and statuses understandable
UX-04	Zero balance	State preserved and read-only/export remains available

18. Implementation sequence and work packages

Phase A — Commercial and scope decisions

Confirm launch jurisdictions, adult/content policy, exact supported game subset, payment model approval, tax handling, credit/refund terms, storage retention, and initial price version. Select a small cohort. Exit: the company can explain exactly what is sold, who can spend, who can refund, and which rights apply.

Phase B — Durable platform foundation

Provision staging/production infrastructure as code. Add identity integration, campaign membership, private asset storage, production composition, secrets, CI, audit correlation, and reliable save/resume. Fix expired-grant recovery, resource attempt identity, durable asset lookup, and oversized-manifest recovery before money is exposed. Exit: hosted non-paying groups can play, leave, resume, and export without developer-only access.

Phase C — Money in test mode

Implement purchases, signed webhooks, source funding lots, double-entry ledger, consent, allocations, quotes, reservations, supplier exposure, settlement, refunds, dispute states, and daily reconciliation. Build billing pages against the real test-mode backend. Exit: all payment/pot/job failure tests pass, including concurrent exhaustion and unknown supplier outcomes.

Phase D — Full hosted user journey

Connect invite/lobby/character/TV flows, pricing disclosure, in-play controls, active-time metering, session summaries, storage management, and customer support. Load-test representative party sizes and resource density. Exit: a full session can be completed, settled, resumed, and reconciled without engineering intervention.

Phase E — Controlled paid pilot

Enable a small invited cohort only after payment/legal/content gates. Keep low exposure caps, manual top-ups, video off, founder observation, and readily available support. Review costs after each session. Exit: finance reconciles, repeated use is demonstrated, failures are rare and recoverable, and the selected price covers measured costs.

Phase F — Wider release

Add capacity and resilience based on measured bottlenecks, not registered-user count. Introduce optional auto-reload only after manual payments are stable. Defer subscriptions and marketplace functions until they solve an observed retention/revenue problem.

Workstream ownership

Workstream	Primary artifacts	Dependencies
Commercial	Approved offer, processor response, terms, rights register	Owner, payment provider, counsel/ accountant
Identity/ product	Auth, memberships, campaigns, page system	Commercial scope; platform foundation
Finance	Ledger schema, state machines, invariants, reconciliation	Price/tax/refund decisions
Runtime	Production server, owner fencing, budget admission	Auth and finance contracts
Assets/saves	Storage catalog, uploads, exports, checkpoint migration	Audience model and runtime versions
Infrastructure	IaC, deployment, backups, alerts, load tests	Runtime and persistence contracts
Quality	Independent adversarial tests and real group acceptance	Integrated builds, not isolated branches

Agents can work in parallel on nonconflicting modules, but shared money types, database migrations, protobuf contracts, and price/consent semantics need one responsible owner. Each task brief must include input revision, permitted files, success/failure behavior, test evidence, and rollback considerations.

There is no reliable calendar estimate until the first integrated finance/runtime slice passes. Track milestones by accepted proof rather than committing to a date based on commits per day. The first paid milestone is a bounded trustworthy product, not completion of every roadmap subsystem.

19. Launch decision and operating scorecard

Must be true before accepting real money

The business model is approved by the processor; necessary terms and tax treatment are reviewed; all paid content/provider rights are covered; identity and membership are real rather than local development credentials; every paid dispatch is reserved; webhook/refund idempotency works; private assets stay private; saves resume after long inactivity; money cannot be rolled back with a game save; backups restore; and support can trace a charge without reading unrelated private roleplay.

Publish only the capability level proved in the tested build. Do not advertise full 2024 rules coverage, six-player reliability, instantaneous responses, infinite storage, or a production SLA merely because code exists for them.

Operating scorecard

Metric	Initial decision rule
Financial invariant errors	Zero tolerated; stop affected paid admission immediately
Provider cost per successful table-hour	Use p50/p95 and waste ratio to tune prices and defaults
End-to-end session completion	Target at least 95% without intervention before expanding cohort
Returning groups	Track actual voluntary second sessions within 30 days
Support cost	Measure minutes; replace the placeholder $1/session reserve
Refund/dispute trend	Investigate causes and acquisition sources, not just totals
Latency and blocked time	Reduce serial work before adding infrastructure blindly
Storage growth	Track retained bytes and transfer per table-hour and per campaign
Backup/recovery	Regular restore evidence, not only successful backup-job status
Unit contribution	Positive after provider waste, payments, support, refunds, and variable hosting

First owner decisions still required

Accept or modify the proposed $2/table-hour + 1.5× AI price test; select the approved funding structure; choose the initial US availability and age/content boundaries; approve the refund and retention promises; verify DigitalOcean database eligibility; choose the pilot infrastructure tier; and set maximum company exposure to uncertain provider jobs and disputed purchases.

Until approved, keep these as explicit configuration decisions—not undocumented assumptions in code. No cloud resource, subscription, processor application, or customer payment has been created by this plan.

20. Source register and evidence boundaries

Sources were checked for this planning pass. Reconfirm vendor prices and account eligibility at purchase. Repository documents may contain historical assumptions; they are not provider invoices or legal determinations.

Vendor and technical sources

- **S1 — DigitalOcean Droplet pricing.** Basic 4 GiB/2 vCPU and 8 GiB/4 vCPU list prices. https://www.digitalocean.com/pricing/droplets
- **S2 — DigitalOcean PostgreSQL pricing.** Standard and Advanced nodes, standby charges, storage, and plan-change notice. https://docs.digitalocean.com/products/databases/postgresql/details/pricing/
- **S3 — DigitalOcean Spaces pricing.** Included storage/transfer and overage rates. https://docs.digitalocean.com/products/spaces/details/pricing/
- **S4 — PostgreSQL/MySQL plan changes.** Account-history cutoffs and treatment of existing clusters. https://docs.digitalocean.com/release-notes/upcoming/dbaas-plan-changes/
- **S5 — DigitalOcean backup pricing.** Percentage-based daily and weekly plans. https://docs.digitalocean.com/products/backups/details/pricing/
- **S6 — Stripe service terms.** Billing-credit restrictions. https://stripe.com/legal/ssa-service-terms
- **S7 — Stripe restricted-business policy.** Stored value, credits, and account approval. https://stripe.com/legal/restricted-businesses
- **S8 — Stripe pricing.** Domestic-card baseline used for the illustrative processing model. https://stripe.com/pricing
- **S9 — Stripe webhooks and Checkout fulfillment.** Signature validation, asynchronous fulfillment, duplicate and unordered events. https://docs.stripe.com/webhooks ; https://docs.stripe.com/checkout/fulfillment
- **S10 — WorkOS pricing.** User management free-tier terms and paid product boundaries. https://workos.com/pricing
- **S11 — WorkOS hosted AuthKit.** Sign-in, sign-up, verification, recovery, and MFA flow support. https://workos.com/docs/authkit/hosted-ui
- **S12 — Spaces private access and CDN limitations.** https://docs.digitalocean.com/products/spaces/how-to/set-file-permissions/ ; https://docs.digitalocean.com/products/spaces/details/limits/
- **S13 — Stripe refunds.** Original payment method and asynchronous refund behavior. https://docs.stripe.com/refunds
- **S14 — Stripe saved-payment methods.** Future-use consent and setup. https://docs.stripe.com/payments/save-and-reuse ; https://docs.stripe.com/payments/save-during-payment
- **S15 — PostgreSQL backup recovery.** Daily backups and seven-day recovery window. https://docs.digitalocean.com/products/databases/postgresql/how-to/restore-from-backups/

- **S16 — Stripe refund fees.** Original processing fees generally not returned. https://support.stripe.com/questions/understanding-fees-for-refunded-payments
- **S17 — DigitalOcean Load Balancer features.** WebSocket support and idle behavior. https://docs.digitalocean.com/products/networking/load-balancers/details/features/
- **S18 — DigitalOcean Load Balancer pricing.** https://docs.digitalocean.com/products/networking/load-balancers/details/pricing/
- **S19 — Managed PostgreSQL vector support.** https://docs.digitalocean.com/products/vector-databases/postgresql/ ; https://docs.digitalocean.com/products/databases/postgresql/details/supported-extensions/
- **S20 — Droplet billing.** Lifecycle billing and capacity costs. https://docs.digitalocean.com/products/droplets/details/pricing/
- **S21 — Spaces shared-responsibility model.** Encryption and customer security responsibilities. https://www.digitalocean.com/security/shared-responsibility-model-spaces
- **S22 — Managed database shared-responsibility model.** https://www.digitalocean.com/security/shared-responsibility-model-managed-databases
- **S23 — Droplet shared-responsibility model.** Local disk security differs from managed storage. https://www.digitalocean.com/security/shared-responsibility-model-droplets
- **S24 — Spaces S3 compatibility.** Versioning/lifecycle/encryption feature scope. https://docs.digitalocean.com/products/spaces/reference/s3-compatibility/
- **S25 — Official SRD and creator guidance.** https://www.dndbeyond.com/srd ; https://www.dndbeyond.com/creator-faq

Repository evidence

- **R1 — Current README at inspected revision.** Implementation status, runtime boundaries, rules subset, known test crash, and licensing caveat. https://github.com/monstercameron/dungeonfluxGame/blob/7350b151e863b2797d16f3999579f48453cfd5fe/README.md
- **R2 — Commerce spend policy.** Pure proposals, six supplier-exposure scopes, and unknown-liability treatment. https://github.com/monstercameron/dungeonfluxGame/blob/7350b151e863b2797d16f3999579f48453cfd5fe/crates/df-commerce/src/spend.rs
- **R3 — Observed image-testing spend note.** Development observation without a per-table-hour denominator. https://github.com/monstercameron/dungeonfluxGame/blob/7350b151e863b2797d16f3999579f48453cfd5fe/planning/asset-provider-research.md
- **R4 — Server concurrency research.** Repository-reported measurements and bottleneck hypotheses, not independently rerun here. https://github.com/monstercameron/dungeonfluxGame/blob/7350b151e863b2797d16f3999579f48453cfd5fe/planning/server-concurrency-research.md

Final recommendation

Build the smallest **trustworthy** hosted business: an authenticated group can fund one table, understand every charge, complete a useful session, recover it later, and keep its characters and media private. DigitalOcean provides an economical starting stack; the billing ledger, consent model, durable saves, and recovery behavior determine whether it is safe to operate.

Do not let a low hosting bill justify weak financial controls, and do not let elaborate infrastructure delay testing whether groups return and pay. The next implementation milestone should be a fully reconciled, end-to-end hosted session—not merely the homepage or a working checkout button.